8-4 · Streamlit 기반의 챗봇 인터페이스 개발 · 실습 1.5H

챗봇 화면 구현

에코 챗봇으로 뼈대를 익히고 재실행을 눈으로

이 시간의 목표

1.5시간 뒤에는

① rerun_demo.py 를 눌러 보며 재실행을 직접 확인한다

② chat_echo.py 로 말풍선이 쌓이는 에코 챗봇을 만든다

③ 다시 그리는 루프를 뺀 사본으로 대화가 사라지는 것을 본다

④ 사이드바 · 스트리밍 · @st.cache_resource 를 붙여 본다

화면을 직접 눌러 보는 것이 오늘 이해의 핵심입니다 — 코드만 봐서는 남지 않습니다

실습 흐름

일곱 단계 — ~/data-eng/week8 한 폴더에서

앱마다 streamlit run 으로 띄우고, 다음 단계 전에 그 터미널에서 Ctrl+C

1. 준비	2. 재실행 확인	3. 에코 챗봇	4. 일부러 실패	5~7. 확장
venv · pip rag_env.py 복사	rerun_demo.py	chat_echo.py	루프를 뺀 사본	사이드바 · 스트리밍 · 캐시

1단계

폴더와 가상환경을 준비하세요

1. **mkdir -p ~/data-eng/week8 → python3 -m venv .venv**
   source .venv/bin/activate — Windows(WSL2) · macOS · 리눅스 모두 같다

2. **pip install streamlit qdrant-client python-dotenv ...**
   langchain-ollama · langchain-openai 까지 — 설치 약 1분 30초 (실측)

3. **cp ~/data-eng/week7/rag_env.py .**
   OpenAI 경로만 .env 도 복사 — 7주차와 같은 모델 · 임베딩 차원

4. **streamlit --version · python rag_env.py**
   Streamlit, version 1.67.0 · PROVIDER = ollama · 모델 = llama3.2 (실측)

경로 바꾸면

768 · 1536

차원 어긋남

1단계 · .streamlit/config.toml

설정 파일 한 장 — localhost 에만 연다

설정이 없으면

첫 실행에 묻고, 내 PC 주소까지 찍는다 (실측)

- 첫 실행에 Email 입력을 묻는다
- Network URL · External URL 이 찍힌다
- 사설 · 공인 IP 가 화면 공유에 노출

VS

printf ... > .streamlit/config.toml

이 폴더에서 띄우는 앱에만 적용된다

- [server] headless = true
- address = "localhost"
- [browser] gatherUsageStats = false

2단계

재실행을 눈으로 — 누를 때마다 시각이 바뀐다

1. **cat > rerun_demo.py → streamlit run rerun_demo.py**
   터미널에 URL: http://localhost:8501 — 브라우저에서 연다
2. **버튼을 누를 때마다 맨 위 시각이 바뀐다**
   스크립트가 위에서부터 다시 돌았다는 증거
3. **「일반 변수 +1」은 누른 직후에만 1**
   다른 버튼을 누르면 0 — counter = 0 줄부터 다시 시작한다
4. **session_state 는 누른 만큼 쌓인다**
   세 번 누르면 3 (실측)

끝낼 때

그 터미널

Ctrl+C

3단계

chat_echo.py — 뼈대 네 단계가 코드로

뼈대	코드
이력 저장소	if "messages" not in st.session_state:
이력 다시 그리기	for m in st.session_state.messages:
입력 받기	prompt = st.chat_input(...)
사용자 말풍선	with st.chat_message("user"):
답 만들기	answer = "따라 말하기: " + prompt
답 말풍선	with st.chat_message("assistant"):

streamlit run chat_echo.py → 여러 번 입력해도 이전 말풍선이 쌓이면 성공 · role 로 두 종류 말풍선

4단계 — 일부러 실패시키기

루프를 뺀 사본 — 이력은 있는데 화면엔 없다

chat_echo.py — 원본

재실행마다 이력 전체를 다시 그린다



VS

chat_echo_noloop.py — 사본

python3 바꿔 쓰기 · grep -c "^# for m" 이 1



5단계

사이드바와 「대화 지우기」를 붙이세요

1. **with st.sidebar: 설정 · 대화 N개 · 지우기**
   cat > chat_echo.py 로 전체본을 다시 쓴다

2. **두 번 보내면 「대화 2개」, 말풍선은 넷**
   사이드바가 이번 입력을 더하기 전에 그려진다 (실측)

3. **지우기 → messages = [] 뒤 st.rerun()**
   말풍선이 바로 사라진다

4. **st.rerun() 을 빼면**
   지운 뒤에도 다음 입력 때까지 옛 화면이 남는다

보이는 것

위→아래

한 번 돈다

6단계

글자가 흘러나오게 — chat_stream.py

1. **python3 바꿔 쓰기로 chat_echo.py → chat_stream.py**
   stream_text() 를 더하고 답 말풍선 한 줄만 바꾼다

2. **stream_text(text) — 한 글자씩 yield**
   time.sleep(0.03) 으로 한 글자씩 나타난다

3. **grep -n "write_stream" chat_stream.py**
   st.write_stream(stream_text(answer)) 한 줄이 나오면 된다

4. **write_stream 은 전체 문자열을 돌려준다**
   그대로 이력에 넣는다 — 검색 실습 app.py 가 LLM 답을 이렇게 흘린다

같은 방식

LLM 스트리밍에도

7단계

@st.cache_resource — 3초가 한 번만

장식자 있음

처음 3.00초 안팎, 「다시 실행」을 누르면 0.00초

- heavy_setup() 안에 time.sleep(3)
- 두 번째부터는 저장한 결과를 돌려준다
- app.py 는 여기서 모델 · 접속을 한 번만

VS

장식자 줄을 지움

누를 때마다 3초씩 걸린다

- 같은 함수, @st.cache_resource 한 줄만 삭제
- 재실행마다 무거운 준비를 다시
- 되돌려 재 보면 캐시가 하는 일이 분명해진다

확인 포인트

이 1.5시간이 끝날 때 확인할 것

1. Streamlit 1.67.0 · PROVIDER 가 7주차와 같다
2. rerun_demo.py 에서 시각이 바뀌고 session_state 만 쌓인다
3. chat_echo.py 는 쌓이고, 루프를 뺀 사본은 방금 것만 보인다
4. 「대화 2개」에 말풍선 넷 · 지우기가 바로 반영된다
5. cache_demo.py 가 처음 3.00초, 다시 누르면 0.00초

8-4 · 실습 1.5H 마무리

다음은 Qdrant 검색·LLM 연동 실습 (1H)

에코 자리에 검색 → 기준값 → LLM 답을 넣어 실제로 답하는 챗봇을 만듭니다

- ✓ **lecture_kb count 11 먼저 확인**
- ✓ **search_test.py 로 검색만 시험**
- ✓ **threshold.py 로 기준값 0.4 정하기**
- ✓ **app.py — 근거가 약하면 LLM 을 부르지 않는다**

가져갈 것

week8 폴더 · 켜 둔 가상환경